

Stop, Verify, or Explore?

Motivation

- Autoregressive LLM reasoning에서 추가 compute는 accuracy를 올릴 수 있지만, 모든 문제에 동일하게 쓰면 비용이 크게 증가함.
- 질문: initial reasoning 상태를 보고 한 번의 routing 결정으로 compute allocation을 adaptive하게 조절할 수 있는가?
- 핵심 아이디어: initial 답 생성 이후 STOP / VERIFY / SC-3 중 하나만 선택하는 one-step post-hoc routing.

Background & Problem Setting

- 기존 LLM reasoning 개선법: chain-of-thought, self-consistency, verification, majority voting.
- 대부분은 모든 예제에 동일한 추가 compute를 적용하거나 여러 generation을 고정으로 실행.
- 본 프로젝트는 한 번만 routing 결정 을 한다는 점에서 차별화됨.
- 제안하는 action:
 - STOP: initial answer 유지
 - VERIFY: 문제를 다시 풀고 기존 풀이/답을 검토하여 수정
 - SC-3: independent reasoning 2개 추가 생성 후 plurality vote

Research Gap

- 대부분 연구는 extra compute를 무조건 사용할 때의 성능 개선에 집중.
- 적은 compute regime에서 state-aware decision 을 통한 compute allocation은 상대적으로 덜 탐구됨.
- 특히 verification과 self-consistency를 동시에 비교하고, utility 기반 oracle 판단과 learned controller를 함께 평가한 사례가 적음.

System Architecture

- 입력: GSM8K 문제와 정답
- pipeline:
 - i. initial prompt로 initial reasoning 생성
 - ii. initial answer extraction 및 correctness 판단
 - iii. VERIFY prompt 또는 SC-3 prompt로 추가 generation 수행
 - iv. STOP/VERIFY/SC-3 action별 final answer 산출
 - v. Oracle utility 계산 및 controller 학습/평가

Experimental Setup

- Dataset: GSM8K test split
- Models: Qwen/Qwen2.5-1.5B-Instruct, Qwen/Qwen2.5-3B-Instruct
- Generation hyperparameters:
 - temperature = 0.8
 - max_new_tokens_initial = 192
 - max_new_tokens_verify = 256
 - max_new_tokens_sc3 = 192
- 평가 지표:
 - accuracy
 - avg total tokens
 - utility = correct - lambda * cost
 - lambda = 0.1
- baseline:
 - Always STOP, Always VERIFY, Always SC-3
 - Random Router, Length Threshold Router
 - Oracle Router
 - Learned controllers: logistic, random forest, value predictor

Results & Evaluation

Qwen2.5-1.5B, 300 examples

- Always STOP: accuracy 0.253, avg tokens 299.68, utility 0.222
- Always VERIFY: accuracy 0.370, avg tokens 916.16, utility 0.275
- Always SC-3: accuracy 0.287, avg tokens 966.53, utility 0.187
- Oracle Router: accuracy 0.473, avg tokens 435.65, utility 0.428
- Random forest calibrated controller: accuracy 0.430, avg tokens 484.83, utility 0.380

Qwen2.5-3B, 100 examples

- Always STOP: accuracy 0.320, avg tokens 299.87, utility 0.289
- Always VERIFY: accuracy 0.540, avg tokens 912.76, utility 0.446
- Always SC-3: accuracy 0.340, avg tokens 969.57, utility 0.240
- Oracle Router: accuracy 0.600, avg tokens 472.56, utility 0.551
- Best learned controller (value predictor): accuracy 0.540, avg tokens 646.98, utility 0.473

Model comparison (100 examples)

- Qwen2.5-1.5B vs Qwen2.5-3B:
 - STOP acc: 0.290 vs 0.320
 - VERIFY acc: 0.440 vs 0.540
 - Oracle acc: 0.530 vs 0.600
 - Best controller acc: 0.470 vs 0.540
- Larger 모델일수록 VERIFY 기반 compute 사용이 더 효과적임.

Discussion

- Value-of-compute signal 존재 확인:
 - Oracle Router는 Always STOP보다 높은 accuracy와 utility를 달성.
 - VERIFY와 SC-3는 모든 문제에서 필요한 것이 아니라, 일부 문제에서만 유의미한 향상을 제공.
- VERIFY가 SC-3보다 우수하게 나타남:
 - Qwen instruct 모델과 GSM8K에서, re-solve-and-verify 방식이 self-consistency 투표보다 더 안정적.
- Learned controller는 oracle gap을 일부 회복했으나 여전히 Oracle 대비 성능 차이 존재:
 - calibrated random forest와 value predictor가 가장 좋았음.
- multiple models 실험으로 제안 방법의 일반성 확보:
 - 1.5B와 3B 모두 비슷한 경향을 보였으며, 3B에서 더 큰 개선이 관찰됨.

Limitations & Future Work

- Dataset 단일: GSM8K만 사용
- 3B 실험 샘플 수 제한: 100 examples
- Oracle label imbalance: 대부분의 example에서 STOP이 최적
- 추가 feature 미구현: entropy, token log-probability, self-reported confidence
- SC-3 확장 검토 필요: SC-5 이상 또는 다른 voting 방식

Reproducibility

- `python3 scripts/run_rollouts.py --dataset gsm8k --model_name Qwen/Qwen2.5-1.5B-Instruct --max_examples 300 --shuffle --seed 7 --ou`
- `python3 scripts/build_oracle.py --rollout_path outputs/gsm8k_rollouts_qwen300_shuffle_v2.jsonl --lambda_values 0.0 0.05 0.1 0.2 --`
- `python3 scripts/train_controller.py --rollout_path outputs/gsm8k_rollouts_qwen25_3b_100_shuffle_v2.jsonl --lambda_value 0.1 --cont`
- `python3 scripts/evaluate_methods.py --rollout_path outputs/gsm8k_rollouts_qwen300_shuffle_v2.jsonl --controller_path outputs/contr`

References

- Wei et al., "Chain of Thought Prompting" (2022)
- Wang et al., "Self-Consistency in Reasoning" (2023)
- Kojima et al., "Large Language Models are Zero-Shot Reasoners" (2022)
- [Project code and results](#)